π-Former: Succinct Proofs of Transformer Inference via Structured Lookup Attention

Hideaki Takahashi

April 2026

Motivation & Challenge

Setting. A model owner runs $\mathcal{M}_\theta$ on user input $\mathbf{x}$ and returns $\mathbf{y}$. The user wants a cryptographic proof that $\mathbf{y} = \mathcal{M}_\theta(\mathbf{x})$, with short proof and cheap verification.

The catch. A SNARK only natively expresses **additions and multiplications in** $\mathbb{F}$. But a transformer block has:

- **Softmax**: $\exp(\cdot)$ + row-wise division.
- **GeLU**: transcendental.
- **LayerNorm**: $\sqrt{\sigma^2 + \epsilon}$ + division.
- **Dense linear**: one field multiplication per weight.

State of the art (zkLLM, zkGPT) approximates these with large lookup tables. Prover time still **minutes per token** for GPT-scale models.

Our angle

Don't fight the model. **Co-design** the model and the proof system so that every op is natively cheap.

Co-Design in One Slide

We replace each painful operation with a SNARK-friendly twin:

Standard op	$\Rightarrow$	**π-Former op**
softmax attention	$\Rightarrow$	**LLA**: linear attention with lookup $\boldsymbol{\phi}$
GeLU	$\Rightarrow$	learnable additive sub-table lookup
LayerNorm (div, $\sqrt{}$)	$\Rightarrow$	constraint-fused LN, no div / $\sqrt{}$
dense $\mathbf{W}$ (FP)	$\Rightarrow$	ternary $\mathbf{W} \in \{-1, 0, +1\} \cdot \alpha$

End result. Every forward-pass operation becomes

- a low-degree **sumcheck** claim, or
- a **structured table lookup** (Lasso).

No "ZK-unfriendly operation tax" to pay.

Toolbox (One-Line Reminders)

Sumcheck. Reduces $H = \sum_{x \in \{0,1\}^n} f(x)$ to a single evaluation $f(r)$ at random r. Verifier cost $O(n\delta)$, soundness error $\leq n\delta/|\mathbb{F}|$.

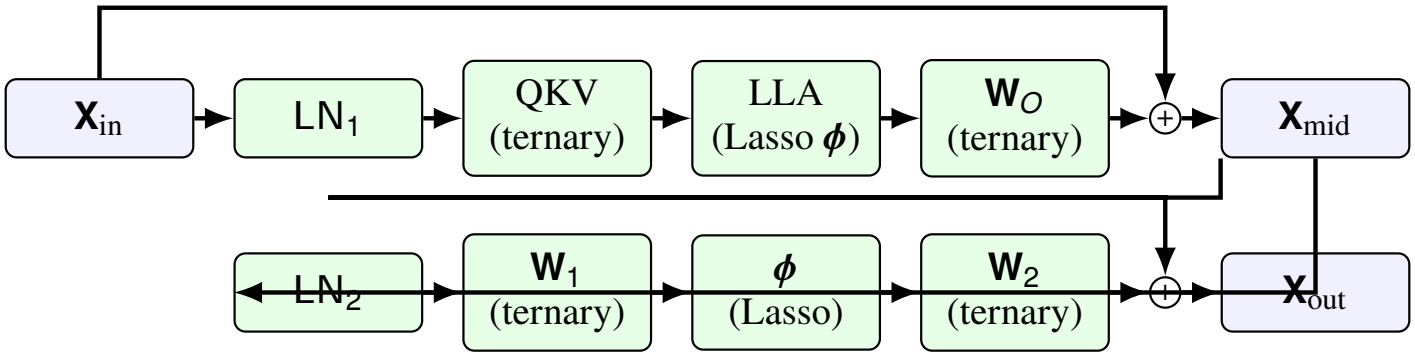
Hyrax PCS. Transparent (no trusted setup) polynomial commitment based on Pedersen / DL on BN254 G1. $O(\sqrt{N})$ commitment size, $O(\sqrt{N})$ open. Importantly: **additively homomorphic**.

Lasso. A lookup argument that proves $v_j = T[\mathrm{idx}_j]$ for many queries against a table T, *cheap when T decomposes into small additive sub-tables*.

Strategy

Build the model so that: attention = sumcheck + Lasso, activation = Lasso, LayerNorm = sumchecks + range checks, projection = sumcheck with no field mults.

The π-Former Block



Color code.

- Green = an op the SNARK proves.
- Ternary boxes have *no* field multiplications inside.
- LLA / $\boldsymbol{\phi}$ are proved via Lasso lookups.
- "+" (residual) is checked by **homomorphic addition of Hyrax commitments** – no proof at all.

Idea 1: Learnable Lookup Attention (LLA)

Linear attention replaces $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})$ by a positive-definite kernel $\kappa(\mathbf{q},\mathbf{k}) = \langle \boldsymbol{\phi}(\mathbf{q}), \boldsymbol{\phi}(\mathbf{k})\rangle$:

$$\mathrm{LLA}(\mathbf{Q},\mathbf{K},\mathbf{V})_t \;=\; \boldsymbol{\phi}(\mathbf{q}_t)^\top \underbrace{\sum_s \boldsymbol{\phi}(\mathbf{k}_s)\mathbf{v}_s^\top}_{=\mathbf{M}\ \text{(context, computed once)}} .$$

Cost: $O(n\,d^2)$ instead of $O(n^2 d)$, no exp, no row division.

Where's $\boldsymbol{\phi}$? We make it learnable and SNARK-friendly:

$$\boldsymbol{\phi}(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\mathrm{int}})\big],$$

i.e. split a b-bit input into c chunks, look each up in a small table T_i of size $2^{b/c}$, sum.

Why this matters.

- For b=16, c=2: two 256-entry tables instead of one 65,536-entry table.
- This is *exactly* the structured-decomposability condition Lasso wants.
- Init so $\sum_i T_i \approx$ GeLU, then train end-to-end with the straight-through estimator.

Idea 2: Ternary Projections

All projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$, LM head) are constrained:

$$\mathbf{W} \in \{-1, 0, +1\}^{m \times n}, \qquad \mathbf{y} = \alpha \cdot (\mathbf{x}\mathbf{W}) + \mathbf{b}$$

with one learnable scalar α per layer (BitNet b1.58 style).

What changes inside the SNARK? Consider the standard sumcheck for a contraction $\mathbf{Y} = \mathbf{X}\mathbf{W}$:

$$\widetilde{\mathbf{Y}}(r_t, r_d) - \mathbf{b}(r_d) \;=\; \alpha \sum_k \widetilde{\mathbf{X}}(r_t, k)\, \widetilde{\mathbf{W}}(k, r_d).$$

Because $\widetilde{\mathbf{W}}(k, r_d) \in \{-1, 0, +1\}$ after evaluation, each prover round is:

"$+= / -= /$ skip" – no field multiplications.

Bonus

The verifying key never holds raw weights – only Hyrax *commitments* to the ternary arrays.

Idea 3: Constraint-Fused LayerNorm

Standard LN has $\sqrt{\sigma^2 + \epsilon}$ and a division – both bad in $\mathbb{F}$. We rewrite the identity as integer relations:

For each row, audit at *one* random index r_t via four checks:

1. **Mean**: degree-2 sumcheck for $\widetilde{\text{sum_x}}(r_t)$.
2. **Variance**: cubic sumcheck on the centered residual.
3. **Sigma range**: prove $\sigma = \lfloor \sqrt{\text{var}}/d \rfloor$ by range-checking $\text{var} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var} \geq 0$.
4. **Y fusion**: a single multi-batched cubic sumcheck for the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs together.

Result. No field divisions, no square roots. Verifier reads γ, β from the VK; everything else is one Hyrax opening per witness.

Cross-Block Batching: The Key Systems Trick

A transformer is the *same protocol, run L times*. So instead of L independent proofs per protocol type, we batch:

Resource	**Folded across blocks into…**
6 sumcheck types	6 *model-level* sumchecks
all Lasso instances	one global multi-Lasso
all LN range proofs	one shared multiplicity commitment
residual +	no proof; just add Hyrax commitments
final openings	10 deferred Hyrax accumulators
	finalised in 1 MSM pair each

The fold. For each protocol type, after the verifier sees all per-block claims H_ℓ, draw $\eta \leftarrow \mathsf{FS}$ and prove

$$H^\star = \sum_{\ell=1}^{L} \eta^{\ell-1} H_\ell$$

with one inner sumcheck. Standard random-linear-combination argument $\Rightarrow$ extra error only $(L-1)/|\mathbb{F}|$.

Soundness Sketch

Cross-block batching (Theorem)

CrossBatch reduces L claims to one with extra error

$$\varepsilon_{\text{batch}} \leq \frac{L-1}{|\mathbb{F}|} + \frac{n\delta}{|\mathbb{F}|}.$$

Why it works. The challenge η is drawn *after* all per-block claims and commitments are absorbed. If any claim is false, $P(\eta) = \sum_\ell \eta^{\ell-1}(\hat{H}_\ell - H_\ell)$ is a non-zero polynomial of degree $\leq L-1$. Schwartz–Zippel: at most $L-1$ bad η over $|\mathbb{F}|$.

End-to-end. A union bound over sumchecks, batched openings, Lasso, and LN sub-proofs gives

$$\varepsilon_{\text{sound}} \leq \frac{m_{\text{sc}}\, n_{\max}\delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}}.$$

For $L \leq 12$, $n_{\max} \leq 64$ over BN254: statistical term $< 2^{-240}$, dominated by the discrete-log term.

Implementation

Two matched halves, one shared integer arithmetic.

- **Rust prover/verifier** (~14k LoC) over BN254.
- **PyTorch training pipeline** with custom `TernaryLinear`, `StructuredLookupActivation`, `LinearAttentionLayer`.
- Same fixed-point quantisation on both sides ⇒ *no train–prove gap*.

Module	**LoC**
`poly/`, `pcs/` (MLE, Hyrax)	~1.4k
`subprotocols/` (sumcheck ×3, GKR)	~1.5k
`lookup/` (Lasso multi, range)	~1.3k
`attention/` (LN, projection, LLA)	~3.2k
`ffn/`, `cross_layer/`	~1.2k
`prover.rs`, `verifier.rs`, CLI	~5.4k
Total	~14k

End-to-End Performance

Single-thread x86_64 CPU. Times are pure compute; IO excluded.

Model	d	L	d_{ff}	T	**Params**	**Prove**	**Verify**	**Proof**
tiny	8	1	16	8	1.1k	0.22 s	0.03 s	108 KB
small	64	2	256	16	133k	1.10 s	0.15 s	1.0 MB
medium	128	4	512	32	924k	**4.24 s**	**0.28 s**	5.5 MB

Headline. 4-layer, ~924k-parameter transformer: **4.2 s prove, 280 ms verify, 5.5 MB proof** on a single CPU core.

Scaling.

- Prove time ~ linear in $L \cdot T \cdot d$ (per-block arithmetic survives batching).
- Verify time *sub-linear*: each cross-block sumcheck is a $\log Td$-round transcript, accumulators finalise in 1 MSM pair each.

Where Does the Verifier Spend Time?

Phase	**tiny**	**small**	**medium**
6 cross-block sumchecks (combined)	<0.2	<0.5	<0.8
Final range / final LN / LM-head	1.7	1.9	3.9
Cross-block batched opens	3.6	44.5	46.6
Global Lasso	1.6	25.6	40.9
Accumulator finalisation (10 MSMs)	13.2	23.7	54.2
Subtotal (ms)	22	99	148

Take-aways.

- Cross-block sumcheck transcripts are *negligible* (< 1 ms) – batching collapses transcript work.
- Hyrax openings + accumulator MSMs dominate.
- These are exactly the inner loops that GPU MSM / parallel sumcheck accelerate.

Comparison, Limitations, Roadmap

System	Attention	Activation
zkLLM	softmax (`tlookup`)	structured lookup
zkGPT	softmax (approx., Plonky2)	lookup table
π-Former	**linear (LLA, exact)**	**learnable** additive lookup

Position. π-Former attention is not a softmax approximation: it is a different (kernelised) attention trained from scratch, so the SNARK proves the model *exactly*. $\boldsymbol{\phi}$ is learnable.

Currently in the SNARK. Single-head; un-normalised LLA numerator; no causal mask; *not* zero-knowledge.

Engineering, not architecture.

- Multi-head: parallel batches of the same sumcheck.
- Causal mask: an indicator polynomial inside the sumcheck.
- ZK: standard blinding layer over committed witnesses.
- GPU MSM + parallel sumcheck: every dominant inner loop is data-parallel.
- Streaming generation: Nova-style folding with $\mathbf{M} = \boldsymbol{\phi}(\mathbf{K})^\top \mathbf{V}$ as the IVC accumulator $\Rightarrow$ constant amortised cost per token.

Take-aways

1. **Co-design beats approximation.** Replace softmax / GeLU / LN / dense $\mathbf{W}$ with twins that are *natively* sumcheck + Lasso.
2. **LLA + ternary + fused LN**: every transformer op has sub-linear verifier cost.
3. **Cross-block batching** folds L per-block proofs into 6 model-level sumchecks, 1 global Lasso, 1 global range, 10 deferred openings.
4. **Result.** 4-layer, 924k-parameter transformer: **4.2 s prove / 280 ms verify / 5.5 MB proof**, single CPU core, transparent setup.

Thank you.

ht2673@columbia.edu